

AssureX Claim Engine

Installation Guide

Group	NN_DevStorm
Batch	2609E2'
Faculty	Sir Minhaj
Repository	https://github.com/Saba1512006/assurex-claim-engine
Live application	https://assurex.pythonanywhere.com
Date	29 September 2026

Student ID	Name	Role
Student1525913	Saba Noor	Team lead · integration, frontend direction, QA, documentation
Student1524867	Muhammad Sami	Backend and ML pipeline developer · deployment
Student1509222	Muhammad Ghanyan	Frontend design (Figma, colour palette, mobile checks)
Student1505677	Muhammad Sami ur Rehman	Testing (sample claims, Chrome and mobile)

Abstract

AssureX Claim Engine evaluates warranty claims for consumer electronics, home appliances and industrial tools. A customer or service center registers a product from its receipt (read by OCR), files a claim with evidence, and the application pre-processes the claim into model features. A calibrated Python classification model (HistGradientBoosting, 89.3% test accuracy on 225 unseen claims) scores the structured record; the same claim is drawn as a Claim Summary Card and classified by a Google Teachable Machine model (93.8% test accuracy). The two predictions and their confidences are compared (Strong Match, Acceptable Match, Weak Match, Model Disagreement, Uncertain Result), the category's warranty policy is executed from JSON configuration, contradictions, missing documents and duplicates are detected, and a decision table produces Likely Valid, Likely Invalid or Manual Review Required. Anything uncertain goes to a human reviewer, and every step is recorded in an audit trail. The application has four roles (customer, service-center staff, claim reviewer, administrator), 295 automated tests and 34 browser tests.

Installation instructions (SRS §1.10 item 9): prerequisites, supported systems, Python version, virtual environment, dependencies, database set-up and seeding, model placement, OCR, environment variables, run and test commands, the default administrator and troubleshooting.

1. Installation instructions

Prerequisites

Item	Requirement
Operating system	Windows 10/11, macOS 12+, Ubuntu 20.04+ (any OS with Python)
Python	3.11 or 3.12 (3.11 recommended). <code>scikit-learn==1.9.1</code> is pinned so the saved model loads, and it is not published for Python 3.10
Git	any recent version
Optional	Tesseract OCR (reads <i>image</i> receipts); Chromium via Playwright (only to re-render report diagrams)

Setup

```
git clone https://github.com/Saba1512006/assurex-claim-engine.git
cd assurex-claim-engine
python -m venv venv
# Windows (PowerShell): .\venv\Scripts\Activate.ps1      macOS/Linux: source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Environment variables

`python database/seed.py` creates a `.env` file with a random `SECRET_KEY` the first time it runs. You can also set these yourself (in `.env` or the environment):

Variable	Default	Purpose
<code>SECRET_KEY</code>	<i>(required)</i>	Signs sessions and CSRF tokens. The app refuses to start without it.
<code>DATABASE_URL</code>	<code>sqlite:///database/assurex.db</code>	Any SQLAlchemy URL, e.g. PostgreSQL
<code>UPLOAD_DIR</code>	<code>data/uploads</code>	Where documents and Claim Summary Cards are stored
<code>WARRANTY_EXPIRY_ALERT_DAYS</code>	<code>30</code>	Initial alert window (admins change it in the app)
<code>RATELIMIT_STORAGE_URI</code>	<code>memory://</code>	Use <code>redis://...</code> when running several workers
<code>FLASK_DEBUG</code>	<code>0</code>	<code>1</code> for the debug server

Database

```
python database/seed.py          # drop + recreate tables, add demo users, products and 12 evaluated claims
python database/seed.py --if-empty # only seed an empty database (used on deploy)
```

Tables are created from the SQLAlchemy models (`src/models/entities.py`). The seed prints the status mix it produced. The **default administrator** is `admin@assurex.local` / `AdminPass123!` — change the password from *Profile* after the first sign-in on any shared deployment, and invite real users from *Access*.

Models

Model	Location	How it gets there
Python classification model	<code>model/python_model/claim_classifier_v2.joblib + model_card_v2.json</code>	Included. Rebuild with <code>python notebooks/train_python_v2.py</code>
Google Teachable Machine	<code>model/teachable_machine/model_unquant.tflite + labels.txt</code>	Train in the browser on <code>data/summary_cards/train/</code> , export <i>Tensorflow Lite</i> → <i>Floating point</i> , then upload in Admin > Models (or copy the two files into the folder). Run the evaluation afterwards.

`labels.txt` may use the canonical names (`0 Valid Claim`) or the folder names (`0 valid, 2 manual_review`).

OCR

PDF receipts with a text layer are read by `pdfplumber` (installed from requirements). To read photos and scans, install Tesseract:

- Windows: install from the UB Mannheim build, keep the default path `C:\Program Files\Tesseract-OCR\`, and add it to `PATH`.
- macOS: `brew install tesseract` · Ubuntu/Debian: `sudo apt install tesseract-ocr`

Without Tesseract, image receipts are still accepted and the user types the details (the app says so).

Run

```
python src/app.py                # development server: http://127.0.0.1:5000
gunicorn wsgi:app --workers 2    # production (Linux/macOS)
python -m pytest -q              # 295 automated tests (+ 34 browser tests: -m e2e
tests/e2e)
```

PythonAnywhere: point the WSGI file at the project and `from wsgi import application`; set `SECRET_KEY` in the WSGI file or a `.env`, then run `python database/seed.py --if-empty` once in a console. Use a **Python 3.11** web app and virtualenv: the saved Python model needs `scikit-learn==1.9.1`, which is not published for Python 3.10. On Linux the image model needs `ai-edge-litert` or `tflite-runtime`. Free accounts have a 512 MB disk quota; install with `pip install --no-cache-dir` and keep `numpy<2` if the system packages were built against NumPy 1.x. Render: `render.yaml` builds, seeds an empty database and generates `SECRET_KEY` automatically.

Folder setup

Everything needed is in the repository. Created at runtime: `data/uploads/` (documents, cards), `database/assurex.db`, `.env`. None of them are committed.

Troubleshooting

Symptom	Fix
<code>RuntimeError: SECRET_KEY is not set</code>	Run <code>python database/seed.py</code> once, or set <code>SECRET_KEY</code>
Sign-in works but you're immediately signed out (local http)	Use <code>python src/app.py</code> (it disables the secure-cookie flag for http). Behind https nothing needs changing.
"Python model could not be loaded" / models <i>Unavailable</i> on the live site	Install the pinned <code>scikit-learn==1.9.1</code> (Python 3.11+), or retrain with <code>python notebooks/train_python_v2.py</code>

Symptom	Fix
Every claim ends in Manual Review	The Teachable Machine model isn't installed yet (Admin > Models shows the status)
"No TensorFlow Lite runtime installed"	<code>pip install ai-edge-litert</code> (or <code>tflite-runtime</code> / <code>tensorflow</code>)
Receipt photo: "no text found"	Install Tesseract or upload a PDF; you can also enter the details by hand
<code>sqlite3.OperationalError: no such column</code> after pulling	The schema changed: <code>python database/seed.py</code> (demo) or migrate your data (see <code>src/security/MIGRATION.md</code>)
"This form expired"	The CSRF token is tied to your session — reload the page and submit again
Too many attempts / account locked	Wait 1 minute (the sign-in page shows a countdown), or an admin uses <i>Sign out</i> on the Access page to clear the lock